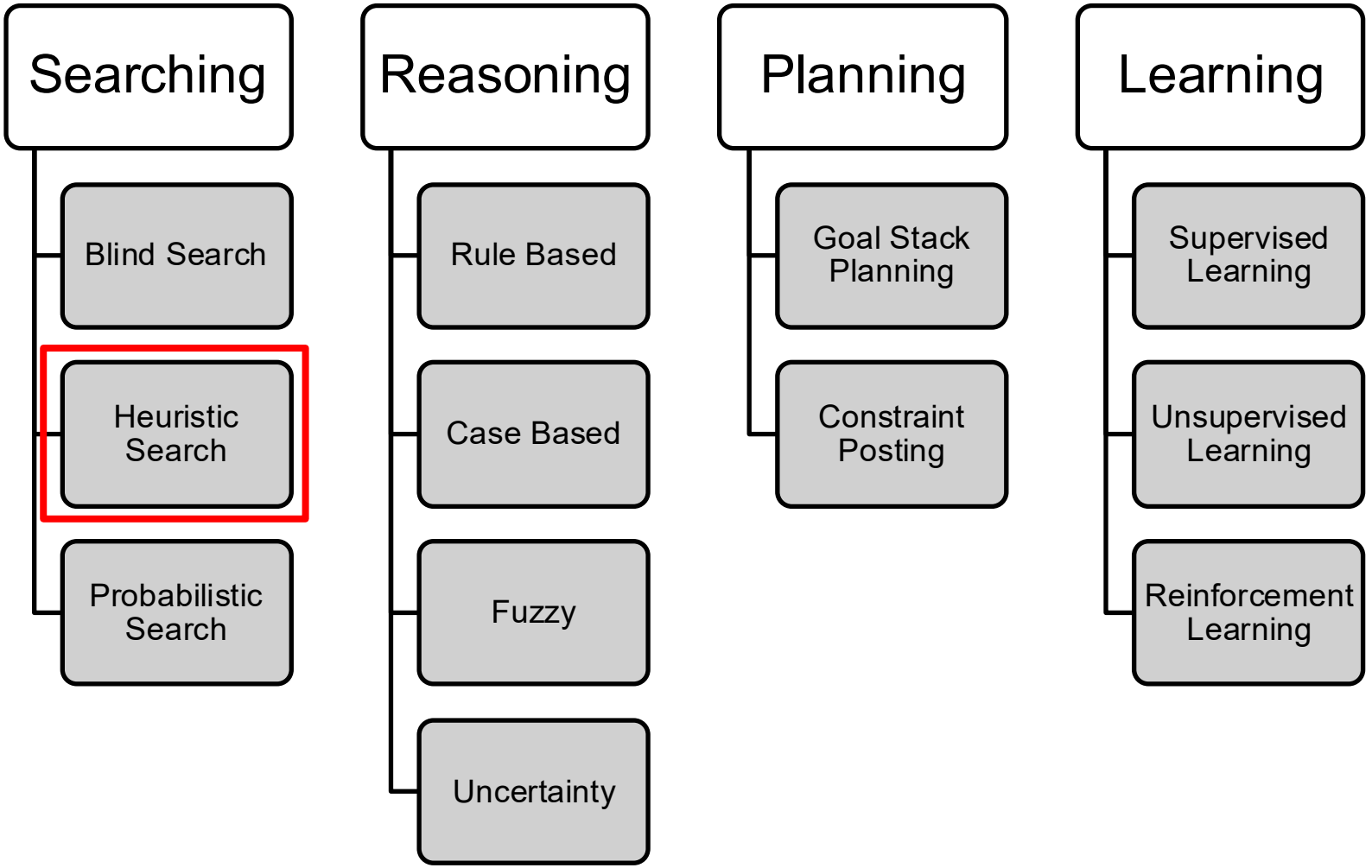
Kecerdasan Buatan
Anatomi Pencarian Heuristik (Informed Search)

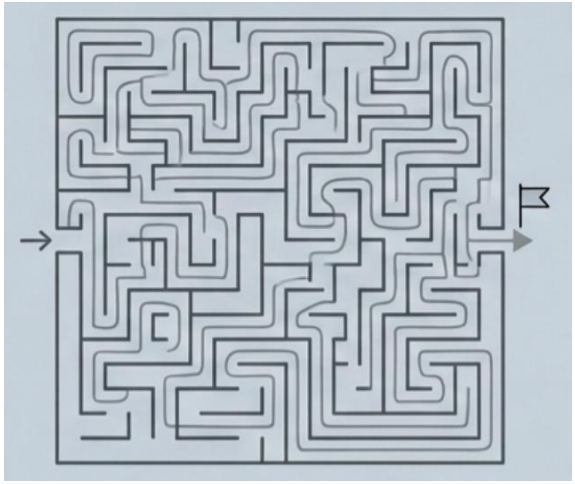


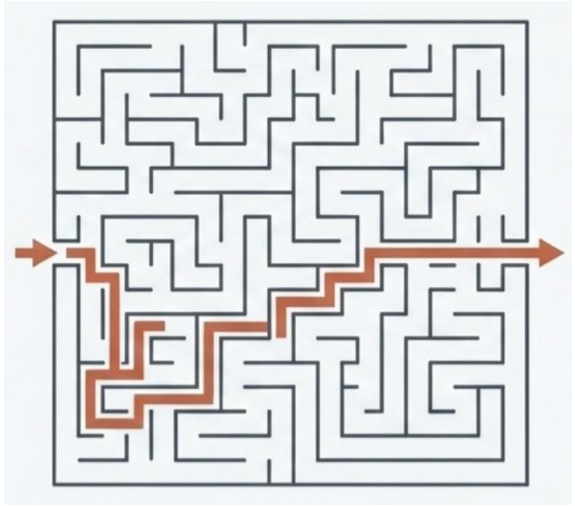
fppt.com

4 Pilar Mesin AI (Teknik Dasar)



Heuristic Search



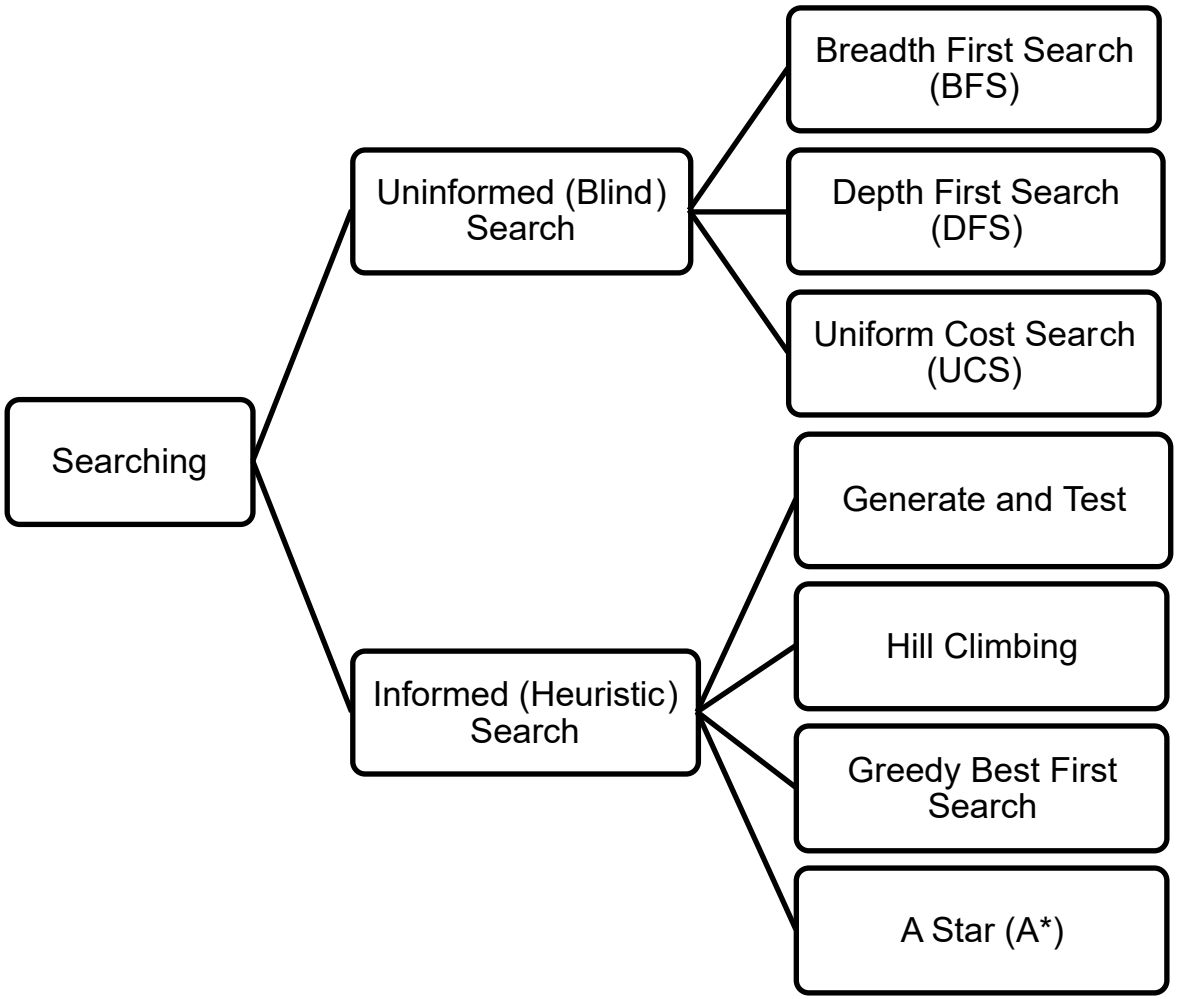


Asal Usul Definisi

Kata Heuristic berasal dari sebuah kata kerja bahasa Yunani, heuriskein, yang berarti ‘mencari’ atau menemukan.

Dalam dunia pemrograman, sebagian orang menggunakan kata heuristik sebagai lawan kata algoritmik, dimana kata heuristik ini diartikan sebagai suatu proses yang dapat menyelesaikan suatu masalah tetapi tidak ada jaminan bahwa solusi yang dicari selalu dapat ditemukan.

Pencarian (Searching)



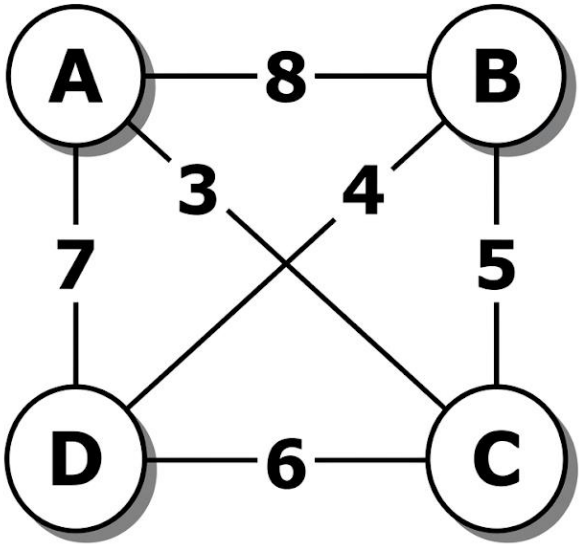
Generate and Test

- Pengabungan antara depth first search dengan pelacakan mundur (backtracking)
- Nilai Pengujian berupa jawaban ‘ya’ atau ‘tidak’
- Jika pembangkit possible solution dikerjakan secara sistimatis, maka prosedur akan mencari solusinya, jika ada.

Generate and Test (Algoritma)

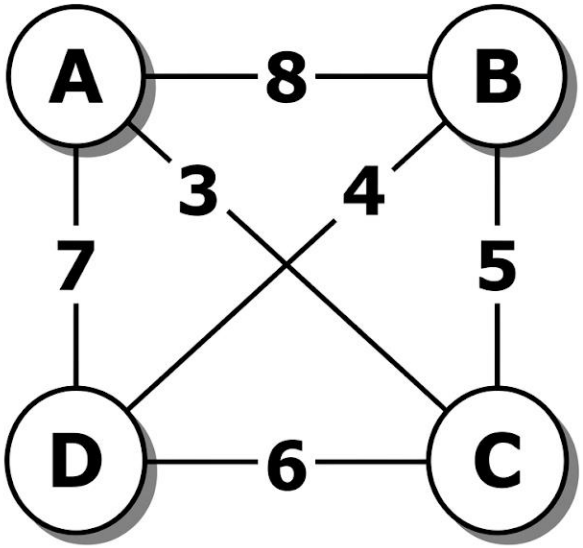
1. Bangkitkan suatu kemungkinan solusi (membangkitkan suatu titik tertentu atau lintasan tertentu dari keadaan awal)
2. Uji apakah node tersebut adalah solusi dengan membandingkan node tersebut atau node akhir dari lintasan yang dipilih dengan kumpulan tujuan yang diharapkan
3. Jika solusi ditemukan,langkah–langkah tersebut dihentikan, jika tidak, kembali ke langkah pertama
4. Jika pembangkitan atau pembuatan solusi–solusi yang dimungkinkan dapat dilakukan secara sistematis, maka prosedur ini akan dapat segera menemukan solusinya, (bila ada).
5. Namun, jika ruang problem sangat besar, maka proses ini akan membutuhkan waktu yang lama, sehingga metode *generate and test ini kurang efisien* untuk masalah yang besar atau kompleks.

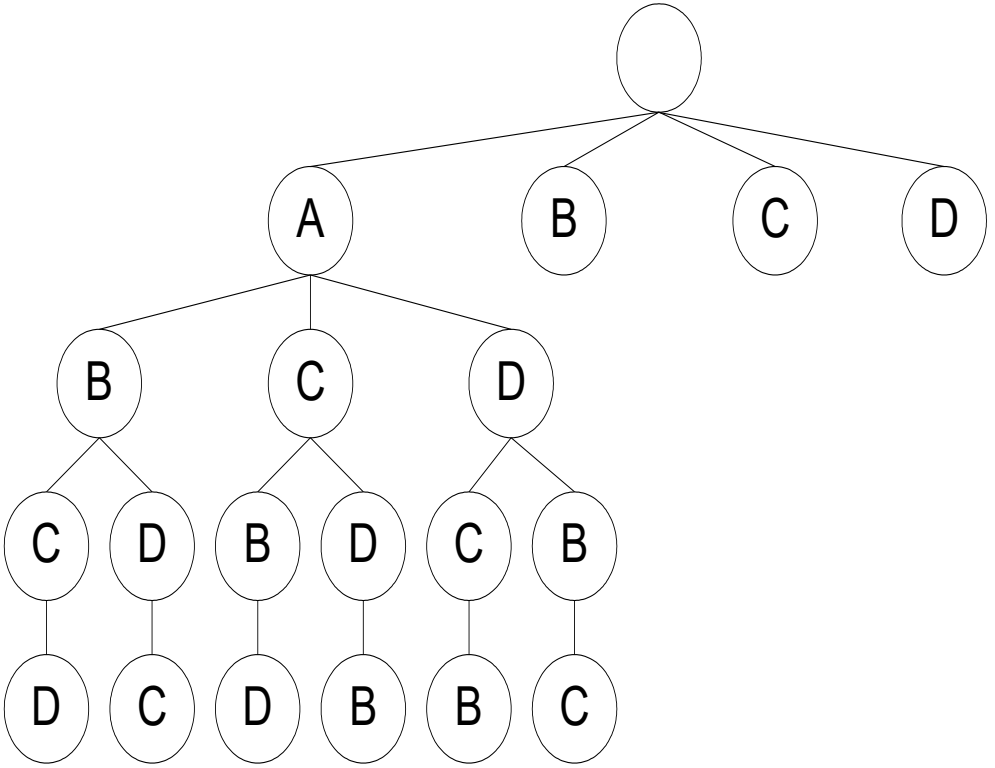
Studi Kasus 1



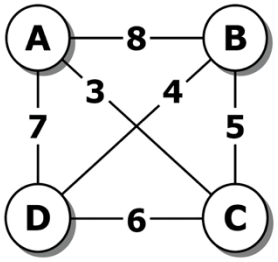
Seorang salesman ingin mengunjungi n kota. Jarak tiap kota sudah diketahui seperti terlihat pada gambar. Bantu salesman untuk menentukan rute terpendek, Dimana setiap kota hanya boleh dikunjungi 1 kali.

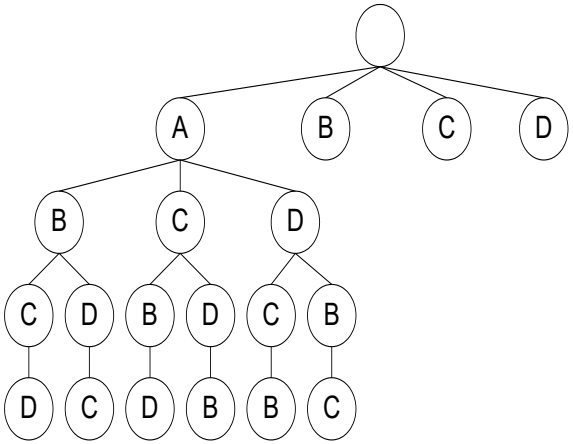
Studi Kasus (Solution)





Studi Kasus (Solution)





Pencarian ke-	Lintasan	Panjang Lintasan	Lintasan Terpilih	Panjang Lintasan Terpilih
1	ABCD	19	ABCD	19
2	ABDC	18	ABDC	18
3	ACBD	12	ACBD	12
4	ACDB	13	ACBD	12
5	ADBC	16	ACBD	12
6	ADCB	18	ACBD	12
7	BACD	17	ACBD	12
8	BADC	21	ACBD	12
9	BCAD	15	ACBD	12
10	BCDA	18	ACBD	12
11	BDAC	14	ACBD	12
…	…	…	…	…
24	DCBA	19	ACBD / DBCA	12

Greedy Best First Search

- Algoritma ini merupakan jenis algoritma Best First Search yang paling sederhana
- Algoritma ini hanya memperhitungkan biaya perkiraan saja
  f(n) = h'(n)
- Karena hanya memperhitungkan biaya perkiraan yang belum tentu kebenarannya, maka algoritma ini menjadi tidak optimal

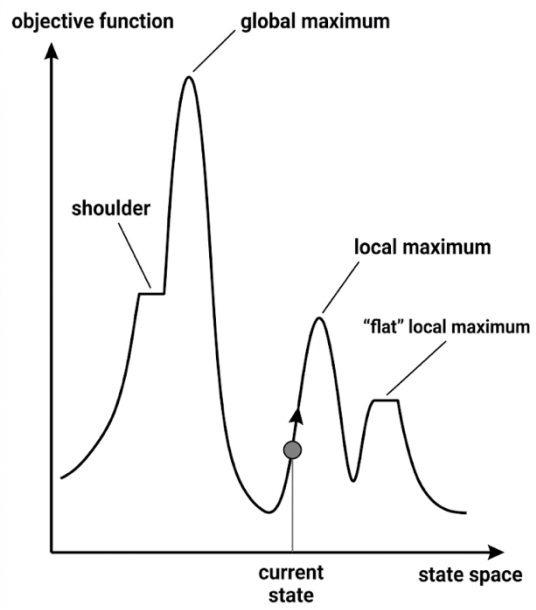
Hill Climbing

- Hill climbing merupakan salah satu variasi metode Generate and Test dimana umpan balik yang berasal dari prosedur uji digunakan untuk memutuskan arah gerak dalam ruang pencarian.
- Dalam prosedur Hill Climbing, fungsi uji dikombinasikan dengan fungsi heuristik yang menyediakan pengukuran kedekatan suatu keadaan yang diberikan dengan tujuan (goal).
- Terdapat beberapa variasi Hill Climbing:
  - Simple Hill Climbing
  - Steepest-Ascent Hill Climbing

Simple Hill Climbing (Algoritma)

1. Mulai keadaan awal, lakukan pengujian: jika tujuan maka stop, jika tidak maka lanjutkan dengan keadaan sekarang sebagai keadaan awal.
2. Ulangi langkah berikutnya hingga solusi ditemukan atau sampai tidak ada operator baru yang diaplikasikan pada keadaan sekarang:
   a. Pilih operator yang belum pernah digunakan, gunakan operator untuk mendapatkan keadaan yang baru
   b. Evaluasi keadaan baru tersebut :
      i. Jika keadaan baru adalah tujuan, keluar
      ii. Jika tidak, namun nilainya lebih baik dari keadaan sekarang, maka jadikan keadaan baru tersebut menjadi keadaan sekarang
      iii. Jika keadaan baru tidak lebih baik daripada keadaan sekarang, maka lanjutkan iterasi
   c. Jika keadaan baru tidak lebih baik daripada keadaan sekarang, maka lanjutkan iterasi

Simple Hill Climbing



Masalah yang akan timbul pada prosedur Hill Climbing :

1. Local optimum adalah suatu keadaan yang lebih baik daripada semua tetangganya namun masih belum lebih baik dari suatu keadaan lain yang jauh letaknya darinya
2. Sering muncul ketika sudah mendekati solusi.
3. Plateau (daratan) adalah suatu daerah datar dari ruang pencarian (search) Dimana keadaan semua tetangga sama dengan keadaan dirinya
4. Ridengane (punggung) : local optimum yang lebih disebabkan karena ketidakmampuan untuk menggunakan 2 operator sekaligus.

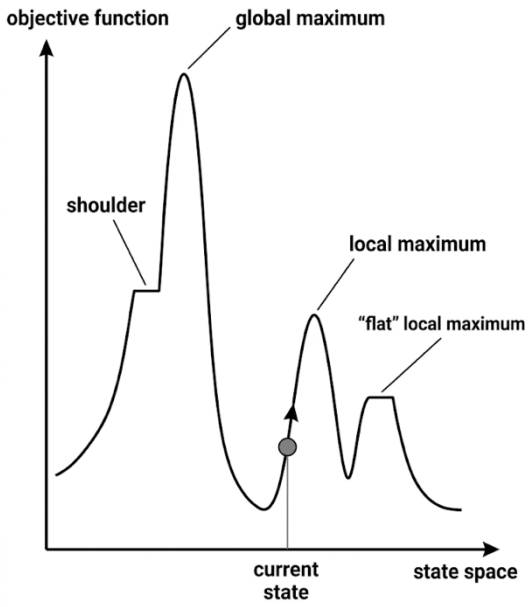
Solusinya:

1. Melakukan langkah balik (backtracking) ke simpul yang lebih awal dan mencoba bergerak ke arah yang lain.
2. Melakukan lompatan besar ke suatu arah untuk mencoba bagian ruang pencarian yang baru.
3. Menerapkan dua atau lebih aturan sebelum melakukan uji coba. Ini bersesuaian dengan bergerak ke beberapa arah sekaligus.

Steepest-Ascent Hill Climbing (Algoritma)

1. Mulai dari keadaan awal, lakukan pengujian: jika merupakan tujuan, maka berhenti; dan jika tidak, lanjutkan dengan keadaan sekarang sebagai keadaan awal.
2. Kerjakan hingga tujuan tercapai atau hingga iterasi tidak memberikan perubahan pada keadaan sekarang.
   a. Tentukan SUCC sebagai nilai heuristik terbaik dari successor-successor.
   b. Kerjakan untuk tiap operator yang digunakan oleh keadaan sekarang:
      i. Gunakan operator tersebut dan bentuk keadaan baru.
      ii. Evaluasi keadaan baru tersebut. Jika merupakan tujuan, keluar. Jika bukan, bandingkan nilai heuristiknya dengan SUCC. Jika lebih baik, jadikan nilai heuristic keadaan baru tersebut sebagai SUCC. Namun jika tidak lebih baik, nilai SUCC tidak berubah.
   c. Jika SUCC lebih baik daripada nilai heuristik keadaan sekarang, ubah node SUCC menjadi keadaan sekarang.

Steepest-Ascent Hill Climbing



Pada Steepest-Ascent Hill Climbing ini, ada 3 masalah yang mungkin, yaitu:

1. Local optimum: keadaan semua tetangga lebih buruk atau sama dengan keadaan dirinya.
2. Plateau: keadaan semua tetangga sama dengan keadaan dirinya.
3. Ridengane: local optimum yang lebih disebabkan karena ketidakmampuan untuk menggunakan 2 operator sekaligus.

A* (A Star)

- Berbeda dengan Greedy, algoritma ini akan **menghitung fungsi heuristic dengan cara menambahkan biaya sebenarnya dengan biaya perkiraan**.
- Heuristik adalah kriteria, metoda, atau prinsip-prinsip untuk menentukan pilihan sejumlah alternatif untuk mencapai sasaran dengan efektif.
- Nilai heuristik dipergunakan untuk mempersempit ruang pencarian
- Dengan heuristik yang benar, maka A* pasti akan mendapatkan solusi (jika memang ada solusinya) yang dicari.
- Dengan kata lain, heuristik adalah fungsi optimasi yang menjadikan algoritma A* lebih baik dari pada algoritma lainnya.
- Namun heuristik masih merupakan estimasi / perkiraan biasa saja,sama sekali tidak ada rumus khususnya. Artinya, setiap kasus memiliki fungsi heuristik yang berbeda-beda.
- Algoritma A* ini bisa dikatakan mirip dengan algoritma Dijkstra, namun pada algoritma Dijkstra, nilai fungsi heuristiknya selalu 0 (nol) sehingga tidak ada fungsi yang mempermudah pencarian solusinya.

A* (A Star)

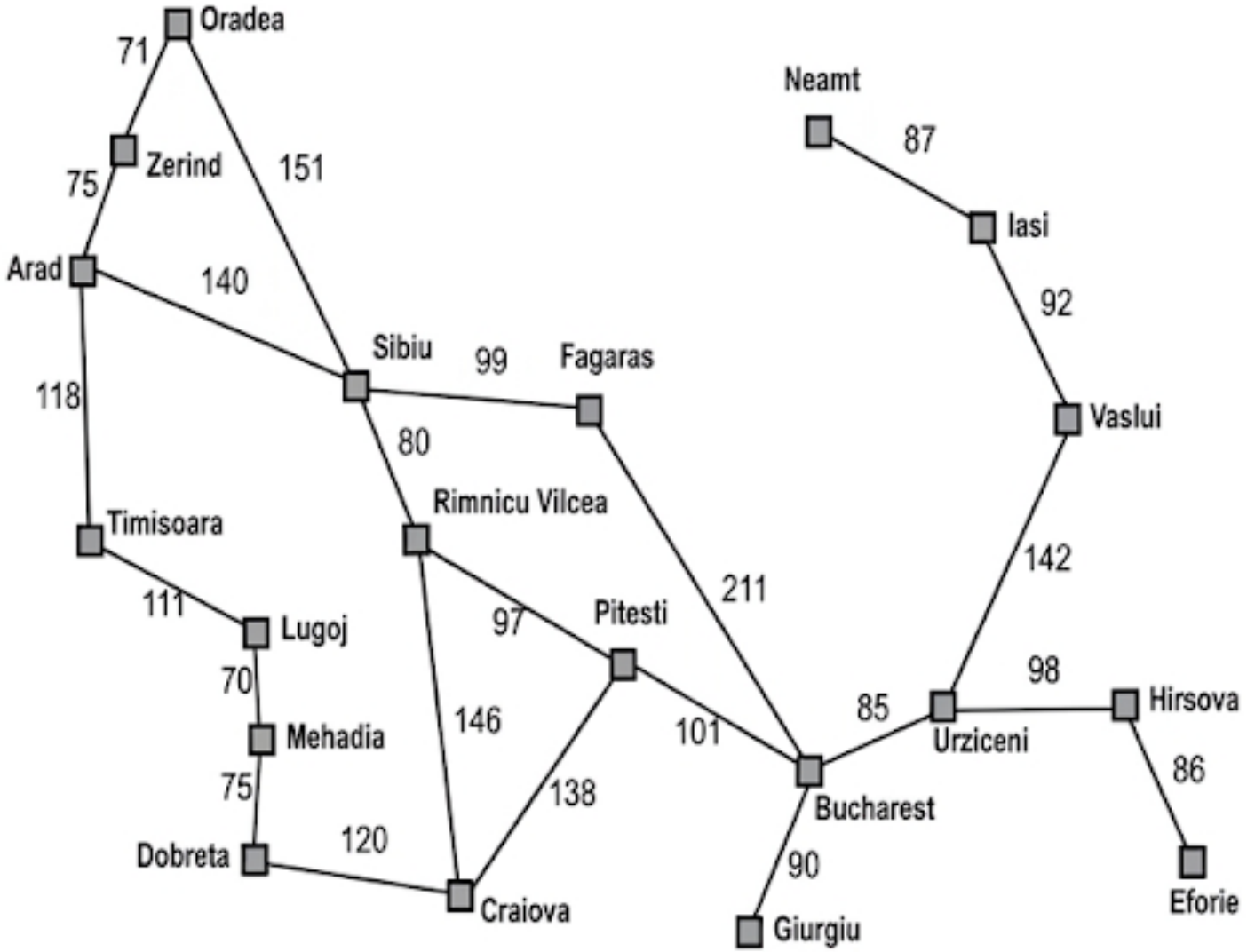
- Metode ini berdasarkan formula fungsi evaluasi:
  f(n) = g(n) + h(n)
- Keterangan:
  - h(n) = biaya estimasi dari node n ke tujuan.
  - g(n) = biaya path / perjalananuntuk menuju n
  - f(n) = solusi biaya estimasi termurah node n untuk mencapai tujuan.
- Membutuhkan 2 antrian : OPEN dan CLOSED

A* (A Star) (Algoritma)

Masukan node awal ke open list. Loop Langkah – langkah di bawah ini :

1. Cari node (n) dengan nilai f(n) yang paling rendah dalam open list. node ini sekarang menjadi current node.
2. Keluarkan current node dari openlist dan masukan ke close list
3. Untuk setiap tetangga dari current node lakukan berikut :
   a. Jika tidak dapat dilalui atau sudah ada dalam close list, abaikan.
   b. Jika belum ada di open list. Buat current node parent dari node tetangga ini. Simpan nilai f, g dan h dari node ini.
   c. Jika sudah ada di open list, cek bila node tetangga ini lebih baik, menggunakan nilai g sebagai ukuran. Jika lebih baik ganti parent dari node ini di open list menjadi current node, lalu kalkulasi ulang nilai g dan f dari node ini.

Soal



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Soal

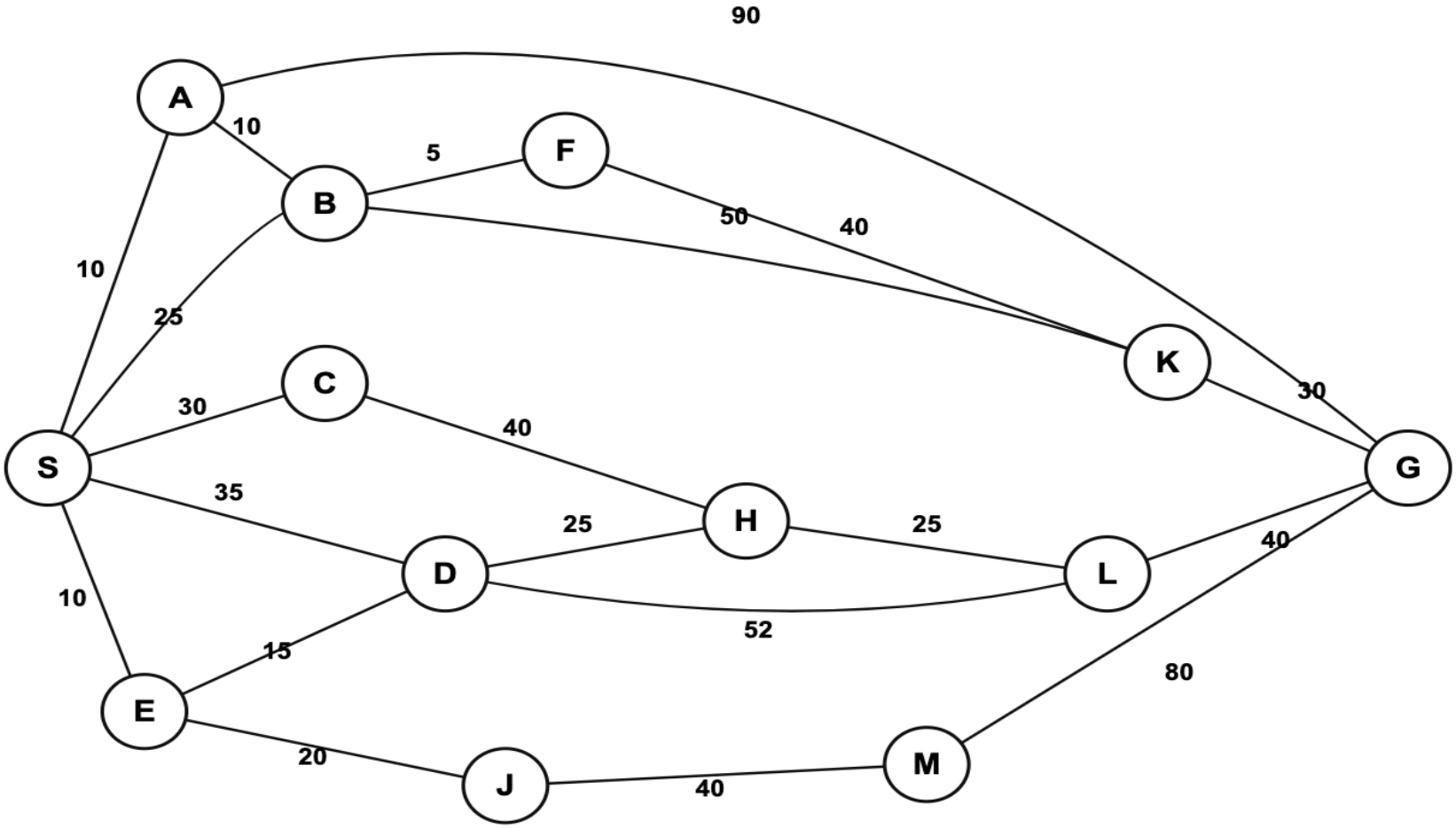
Vlad adalah seorang mahasiswa pascasarjana yang sedang mengikuti pertukaran pelajar di kota Oradea. Ia mendapat kabar mendadak bahwa makalah penelitiannya diterima di sebuah konferensi internasional bergengsi yang diselenggarakan di ibu kota, Bucharest. Waktu sangat berharga, dan ia harus segera berangkat menggunakan mobil sewaannya. Berbekal peta jalan raya Rumania, Vlad mencatat dua informasi penting: jarak tempuh jalan raya antar kota yang terhubung langsung, dan perkiraan jarak udara (garis lurus) dari setiap kota menuju Bucharest yang ditunjukkan oleh sistem GPS-nya. Dari Oradea, Vlad melihat ada dua kota persinggahan awal yang bisa dipilih: Zerind dengan jarak tempuh 71 km, dan Sibiu dengan jarak tempuh 151 km. Sistem GPS-nya menunjukkan bahwa jarak garis lurus dari posisinya di Oradea ke Bucharest adalah 380 km, sementara estimasi dari Zerind adalah 374 km, dan dari Sibiu adalah 253 km. Jika Vlad melanjutkan perjalanan dari Sibiu, jalurnya kembali bercabang menuju Fagaras (jarak tempuh 99 km, estimasi ke Bucharest 176 km), Rimnicu Vilcea (jarak tempuh 80 km, estimasi ke Bucharest 193 km), Arad (jarak tempuh 140 km, estimasi ke Bucharest 366 km), atau memutar kembali ke Oradea. Dari persimpangan-persimpangan tersebut, jalan akan terus bercabang menuju kota-kota lain seperti Pitesti, Craiova, dan akhirnya bermuara di Bucharest.

Soal

Berdasarkan peta dan informasi jarak yang dimiliki Vlad, selesaikan dua instruksi navigasi berikut:

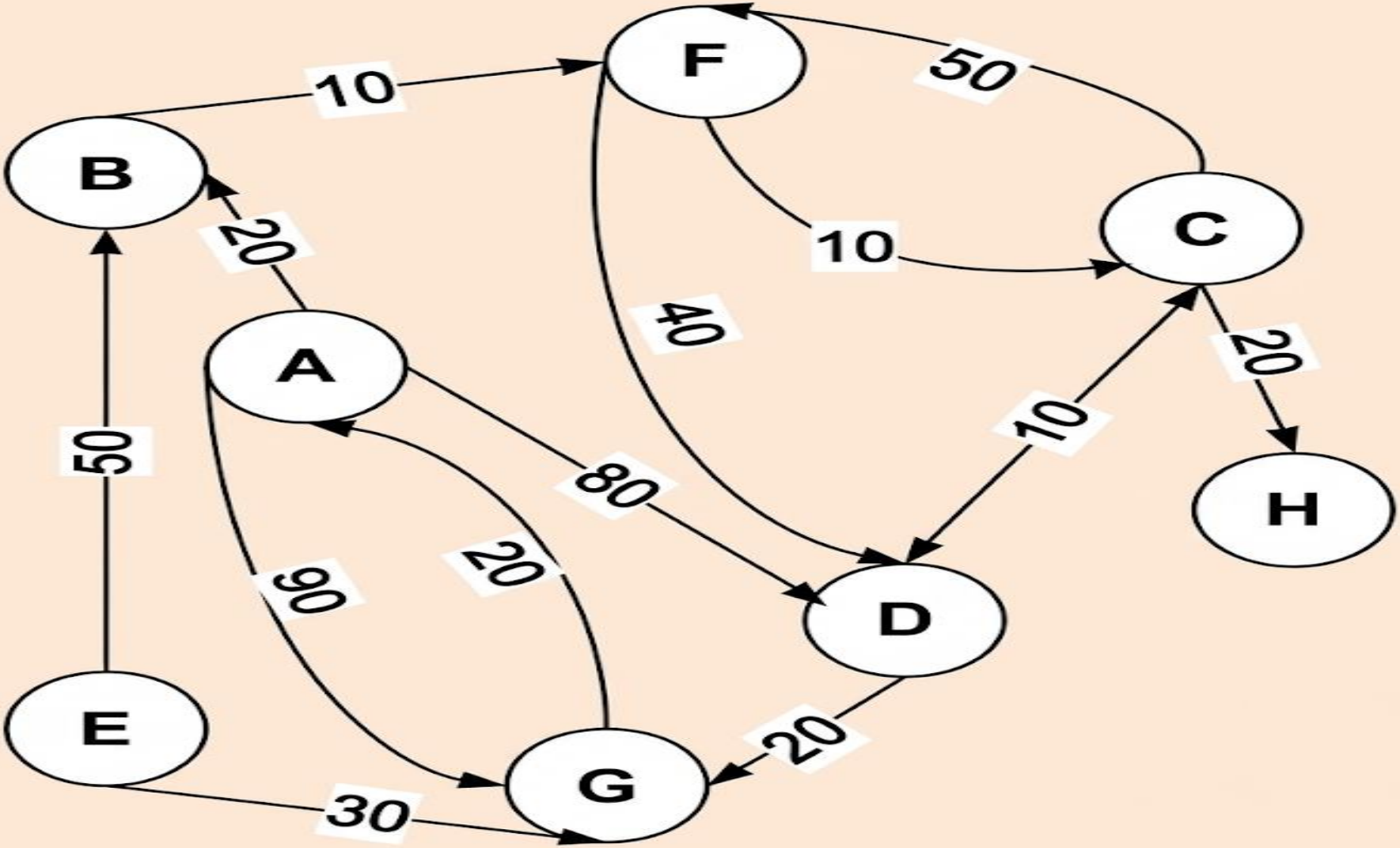
- Penelusuran Steepest-Ascent Hill Climbing. Tentukan rute kota demi kota yang akan dilalui Vlad jika ia mengambil keputusan secara terburu-buru dengan murni hanya memilih kota tetangga yang memiliki estimasi jarak garis lurus terdekat menuju Bucharest, tanpa pernah memperhitungkan bensin dan jarak aktual yang sudah ia tempuh di belakangnya.
- Penelusuran Algoritma *A-Star (A)**. Tentukan rute paling optimal yang harus dipilih Vlad jika ia memikirkan efisiensi bahan bakar secara matang, dengan mengevaluasi setiap rute menggunakan kombinasi antara jarak aktual yang telah ditempuh dari Oradea $g(n)$ditambah estimasi sisa jarak ke Bucharest $h(n)$. Jelaskan perbandingan total biaya jarak aktual antara rute yang dihasilkan algoritma A* ini dengan rute hasil Hill Climbing.

Soal



n	S	A	B	C	D	E	F	G	H	J	K	L	M
h(n)	80	80	60	70	85	74	70	0	40	100	30	20	70

Soal





Selesai

fppt.com